

Week 5 Lab — Advanced Python + NumPy

Write Less, Compute Faster · Total: 35 marks (+3 bonus) · Due: end of Week 5

Name: _____ Student ID: _____ Program: _____

Instructions

- Work in a notebook named **Week5_YourName**. You need **week5_kigali_weather.csv** (60 days of Kigali weather) next to the notebook.
- **Rule of the week:** no for loops in Parts 3–5 (except printing the Bonus). If you're writing a loop, there is an array method or a boolean mask for that.
- **Submit:** (1) completed notebook, (2) screenshots, (3) a ½–1 page report with one insight about Kigali's weather your analysis revealed.

The Dataset — week5_kigali_weather.csv

60 daily records, September–October 2026. Columns: 0 day · 1 date · 2 temp_c · 3 rainfall_mm · 4 humidity_pct

day	date	temp_c	rainfall_mm	humidity_pct
1	2026-09-01	24.4	0.0	77.0
2	2026-09-02	25.9	0.0	74.7
3	2026-09-03	24.5	0.0	72.1
4	2026-09-04	26.0	0.0	76.3
5	2026-09-05	24.6	22.9	73.2
...	... 55 more days ...			

The lab notebook gives you a ready-made loading cell that builds four NumPy arrays: `dates`, `temps`, `rain`, `hum`.

Part 1 · List Comprehensions (6 marks)

```
temps = [24.4, 25.9, 28.6, 19.3, 26.0, 22.1]      # given for Part 1
raw    = ["24.4", "25.9", "28.6", "19.3"]        # given for Ex 1.3
```

Exercise 1.1 — Transform (2 marks)

Convert every temperature to Fahrenheit ($t \times 9/5 + 32$) with a one-line comprehension — no for block.

Exercise 1.2 — Filter (2 marks)

In one line each: (a) `hot` = temperatures ≥ 26 ; (b) `cool_rounded` = temperatures below 24, rounded with `round(t)`.

Exercise 1.3 — Convert types (2 marks)

CSV files give you strings. Convert `raw` to a list of floats with a comprehension, then print its average.

Part 2 · lambda (4 marks)

Exercise 2.1 — Rank with sorted (2 marks)

Given `results = [("Aline", 85), ("Eric", 78), ("Diane", 91), ("Samuel", 92), ("Grace", 76)]`, sort from highest to lowest score with `sorted(..., key=lambda ..., reverse=True)` and print the top 3.

Exercise 2.2 — max with a key (2 marks)

Given `rain_log = [(3, 0.0), (7, 12.5), (11, 3.2), (14, 26.2), (18, 8.9)]` as (day, rainfall_mm) pairs, find the wettest day in one line with `max(..., key=lambda ...)`.

Part 3 · NumPy Arrays (8 marks)

Exercise 3.1 — The speed test (2 marks)

Run the provided speed-test cell (Python loop vs `arr * 2` on one million numbers). In a comment, note roughly how many times faster NumPy was on your machine.

Exercise 3.2 — Describe the temperatures (3 marks)

Using array methods only: print mean, max, min, and std of temps, labeled with f-strings.

Checkpoint: the mean is 24.00 °C.

Exercise 3.3 — The hottest day (3 marks)

`np.argmax(temps)` gives the position of the maximum. Print the DATE of the hottest day and its temperature using `dates[position]`.

Part 4 · Boolean Masking (10 marks)

Exercise 4.1 — Hot days (3 marks)

A hot day is ≥ 26 °C. Using masks: (a) how many hot days? (b) the average temperature on those days?

Exercise 4.2 — Rain report (3 marks)

Using masks on rain: (a) number of rainy days (`rain > 0`), (b) total rainfall over 60 days, (c) heaviest single-day rainfall.

Exercise 4.3 — Cross-array analysis (4 marks)

(a) Compare average temperature on rainy vs dry days — mask from rain, data from temps. (b) Count days that were hot AND rainy — remember `&` with parentheses around each condition. (c) Count days with humidity ≥ 70 .

Part 5 · Anomalies & Trends (7 marks)

Exercise 5.1 — Anomaly detection (4 marks)

Compute `anomaly = temps - temps.mean()` (vectorized). (a) Print the DATES of all days more than 3 °C above the mean — `dates[mask]` works. (b) Count the days more than 3 °C below the mean.

Exercise 5.2 — Is the season changing? (3 marks)

Compare the first 30 days (`temps[:30]`) with the last 30 (`temps[30:]`). Print both averages and state in a comment whether Kigali got warmer or cooler.

Bonus (+3) — Top 5 wettest days

Pair dates with rainfall using `zip(dates, rain)`, rank with `sorted(..., key=lambda ..., reverse=True)`, and print the 5 wettest days with their dates.

Part 6 · Reflection (required, checked for completion)

1. In Week 3 you needed ~15 lines to average a column; today it was `temps.mean()`. What did NumPy hide from you — and why was it still important to learn the loop version first?
2. One insight about Kigali's weather that your analysis revealed.

✓ Submission checklist

- ☐ Notebook renamed Week5_YourName
- ☐ All exercises run without errors
- ☐ No for loops in Parts 3–5
- ☐ Reflection answered
- ☐ Screenshots
- ☐ Short report
- ☐ Submitted before end of Week 5

Marking scheme: Part 1 — 6 · Part 2 — 4 · Part 3 — 8 · Part 4 — 10 · Part 5 — 7 · Total 35 (+3 bonus).